

Project Title

Distributed File Storage Server using AWS

Project ka Main Aim

Humne ek aisa storage system banana tha jo:

- ✓ scalable ho
- ✓ reliable ho
- ✓ fault tolerant ho
- ✓ shared storage provide kare
- ✓ monitoring ho

Traditional single server me agar server down ho gaya to pura system down ho jata hai. Isliye humne AWS pe multiple services use karke distributed architecture banaya.

Step 1: IAM Setup

Sabse pehle humne AWS account pe kaam start kiya.

AWS root account directly use karna secure nahi hota, isliye humne:

AWS Identity and Access Management use kiya.

Kya kiya?

- IAM user create kiya
- Usko permissions di

Why?

Taaki secure login ho aur root account use na karna pade.

Step 2: VPC Setup

Uske baad humne networking setup ki.

Humne Amazon VPC create ki.

Kya kiya?

- Ek VPC create ki
- Public subnet create ki
- Internet Gateway attach kiya
- Route table configure kiya

Why?

Taaki humare EC2 instances internet access kar sakein aur same private network me communicate kar sakein.

Simple language me:

VPC = private network

Subnet = network ka part

IGW = internet ka gate

Route table = traffic ka map

Step 3: Launch EC2 Instances

Uske baad humne 2 Amazon EC2 instances launch kiye.

Why 2?

High availability ke liye.

Agar ek instance down ho jaye to doosra kaam kare.

Kya choose kiya?

- Amazon Linux AMI
 - t2.micro / t3.micro
 - Security Group attach kiya
-

Step 4: Security Group Setup

Security Group ek firewall hota hai.

Humne ports allow kiye:

Port	Use
------	-----

22	SSH
----	-----

80	HTTP
----	------

2049	EFS
------	-----

3306	MySQL
------	-------

Why?

- 22 → terminal access
 - 80 → website access
 - 2049 → EFS mount
 - 3306 → RDS connect
-

Step 5: Apache Setup

Ab humne web server install kiya.

Dono instances pe:

```
sudo yum update -y  
sudo yum install httpd -y
```

Why?

Apache install karne ke liye.

Start kiya:

```
sudo systemctl start httpd
```

Enable kiya:

```
sudo systemctl enable httpd
```

Status check:

```
sudo systemctl status httpd
```

Browser me EC2 IP khola:

<http://<EC2-IP>>

Default Apache page open hui.

Proof:

Web server ready.

Step 6: EFS Setup

Ab shared storage ke liye humne Amazon EFS create kiya.

Kya kiya?

- EFS create ki
 - Same VPC choose ki
 - Mount targets create kiye
 - Security group me 2049 allow kiya
-

EC2 pe install kiya:

```
sudo yum install -y amazon-efs-utils
```

Directory banayi:

```
sudo mkdir /mnt/efs
```

Mount kiya:

```
sudo mount -t efs fs-xxxx:/ /mnt/efs
```

Check kiya:

```
df -h | grep efs
```

Why?

Taaki same storage dono servers use karein.

Step 7: Shared File Test

Instance-1 pe:

```
echo "hello from ec2-user" | sudo tee /mnt/efs/test.txt
```

Instance-2 pe:

```
cat /mnt/efs/test.txt
```

Same output aaya.

Proof:

EFS shared storage working.

Step 8: Apache + EFS Integration

Humne Apache ko EFS se connect kiya.

Purana web root हटाया:

```
sudo rm -rf /var/www/html
```

Symlink banaya:

```
sudo ln -s /mnt/efs /var/www/html
```

Restart:

```
sudo systemctl restart httpd
```

File create ki:

```
echo "Hello from Vishal" | sudo tee /mnt/efs/index.html
```

Browser pe open kiya:

```
http://<EC2-IP>
```

Output aaya.

Why?

Ab browser se EFS ki file serve ho rahi thi.

Step 9: Load Balancer Setup

Humne AWS Application Load Balancer create kiya.

Why?

Traffic distribute karne ke liye.

Kya kiya?

- Internet-facing ALB create kiya
- Listener add kiya
- Target Group create kiya
- Dono instances add kiye

Health check:

/

ALB DNS open kiya.

Step 10: Load Balancing Demo

Humne EFS hata ke local file use ki.

Instance-1:

```
echo "Hello from Instance 1" > /var/www/html/index.html
```

Instance-2:

```
echo "Hello from Instance 2" > /var/www/html/index.html
```

ALB refresh kiya.

Output alternate aaya:

Hello from Instance 1

Hello from Instance 2

Proof:

Load balancing working.

Step 11: RDS Setup

Humne Amazon RDS create kiya.

Engine:

MySQL

EC2 pe install:

```
sudo yum install mariadb105 -y
```

Connect:

```
mysql -h endpoint -u admin -p
```

Database create:

```
CREATE DATABASE storage;
```

Use:

```
USE storage;
```

Table create:

```
CREATE TABLE files(  
id INT AUTO_INCREMENT PRIMARY KEY,  
filename VARCHAR(100)  
);
```

Insert:

```
INSERT INTO files(filename) VALUES ('file1.txt');
```

Why?

Metadata store karne ke liye.

Step 12: PHP Integration

db.php banaya.

Purpose:

Database se data fetch karna.

Browser:

```
http://<ALB-DNS>/db.php
```

Output aaya:

```
file1.txt
```

```
file2.txt
```

upload.php banaya.

Purpose:

Files upload karna.

Step 13: Frontend Deployment

Humne HTML frontend deploy kiya.

File rakhi:

`/var/www/html/index.html`

Browser:

`http://<ALB-DNS>`

Frontend live.

Step 14: CloudWatch Monitoring

Humne Amazon CloudWatch use kiya.

Metrics monitor kiye:

- CPUUtilization
 - NetworkIn
 - NetworkOut
 - RequestCount
-

Alarm create kiya:

CPU > 70%

Test:

`yes > /dev/null &`

CPU spike aaya.

Stop:

`pkill yes`

Final Architecture

User → ALB → EC2-1 / EC2-2 → EFS

↓

RDS

↓

CloudWatch

Benefits

- ✓ High availability
- ✓ Fault tolerance

- ✓ Shared storage
- ✓ Monitoring
- ✓ Centralized DB
- ✓ Scalable architecture